Transformers

The architecture behind GPT, BERT, Claude and modern AI, built up from tokens and attention to a working GPT, Hugging Face models and the techniques used in production.

From the basics to advanced techniques, with explanations and working code.

1. **Foundations**: why Transformers, tokenization, embeddings and positional encoding
2. **Attention**: queries, keys and values, scaling, masking, multi-head attention
3. **The full model**: Transformer blocks, encoders and decoders, a mini GPT in PyTorch
4. **Pre-trained models**: Hugging Face pipelines, contextual embeddings, text generation and sampling, fine-tuning
5. **Advanced**: Vision Transformers, KV cache, efficient attention, RoPE, LoRA

Contents

Part 1: Foundations

1. Why Transformers?
2. The big picture
3. Tokenization
4. Embeddings and positional encoding

Part 2: Attention

5. The intuition
6. Scaled dot-product attention
7. Masking
8. Multi-head attention

Part 3: The full model

9. The Transformer block
10. Encoders, decoders and encoder-decoders
11. Build a mini GPT in PyTorch

Part 4: Using pre-trained models

12. Pipelines: one line per task
13. Contextual embeddings: the same word, different vectors
14. Text generation and sampling strategies
15. Fine-tuning BERT for classification

Part 5: Advanced topics

16. Vision Transformers (ViT)
17. The KV cache: why generation is fast
18. The cost of long context and efficient attention
19. Rotary position embeddings (RoPE)
20. Parameter-efficient fine-tuning: LoRA
21. What makes modern LLMs different

Summary and interview questions

Cheat sheet

Interview questions to practise

Part 1: Foundations

1. Why Transformers?

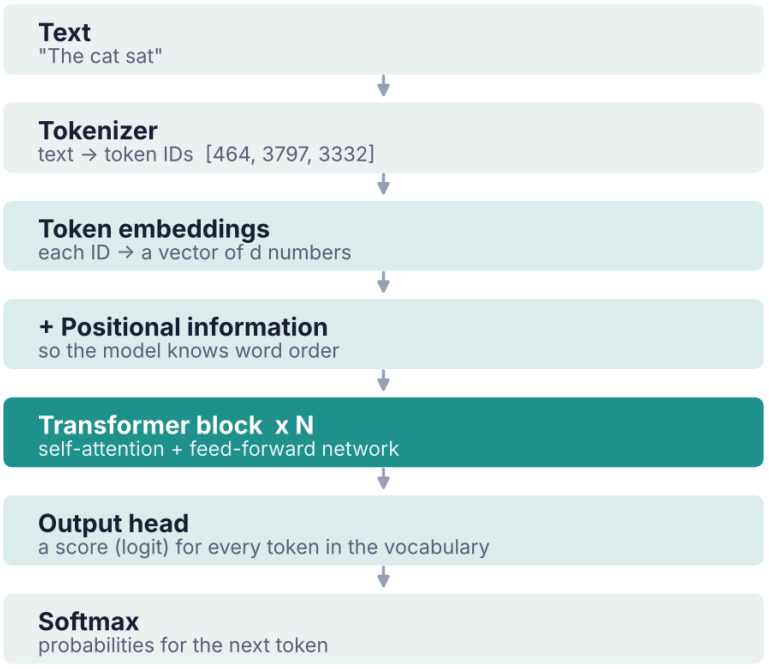
Before 2017, language models mostly used **recurrent neural networks** (RNNs and LSTMs). They read text one word at a time, passing a "memory" vector from step to step. This had two big problems:

- **Slow training.** Step 100 can't start until step 99 is finished, so RNNs can't use the massive parallelism of GPUs.
- **Forgetting.** Information from the start of a long text fades as it passes through hundreds of steps.

The paper *Attention Is All You Need* (Vaswani et al., 2017) replaced recurrence entirely with **attention**: every word looks directly at every other word, in one parallel step. That made it possible to train on enormous datasets, which led to BERT, GPT, T5, and today's large language models. Transformers now also power image models (ViT), speech (Whisper), protein folding (AlphaFold) and more.

2. The big picture

Every Transformer language model follows the same pipeline:



The pipeline of a decoder-only Transformer such as GPT. Everything in this guide explains one of these boxes.

3. Tokenization

Models don't read characters or whole words; they read **tokens**. Modern tokenizers use **subword** units: common words get a single token, while rare words are split into pieces. This keeps the vocabulary manageable (30,000 to 200,000 tokens) while still handling any word, including typos and new names.

The most common algorithm is **Byte-Pair Encoding (BPE)**: start from single characters, then repeatedly merge the most frequent adjacent pair into a new token. Here it is from scratch:

Python

Tested: output below is real

```python
from collections import Counter

# word -> how often it appears; "_" marks the end of a word
word_counts = {"low_": 5, "lower_": 2, "newest_": 6, "widest_": 3}
words = {tuple(w): c for w, c in word_counts.items()}   # start from single characters

def most_frequent_pair(words):
    pairs = Counter()
    for symbols, count in words.items():
        for a, b in zip(symbols, symbols[1:]):
            pairs[(a, b)] += count
    return pairs.most_common(1)[0]

def merge(words, pair):
    merged = {}
    for symbols, count in words.items():
        out, i = [], 0
        while i < len(symbols):
            if i < len(symbols) - 1 and (symbols[i], symbols[i + 1]) == pair:
                out.append(symbols[i] + symbols[i + 1]); i += 2
            else:
                out.append(symbols[i]); i += 1
        merged[tuple(out)] = count
    return merged

merges = []
for step in range(8):
    pair, freq = most_frequent_pair(words)
    words = merge(words, pair)
    merges.append(pair)
    print(f"merge {step + 1}: {pair[0]!r:7} + {pair[1]!r:7} (seen {freq}x)")

print("\nFinal tokens:", [" ".join(s) for s in words])
```

Output

```
merge 1: 'e'     + 's'     (seen 9x)
merge 2: 'es'    + 't'     (seen 9x)
merge 3: 'est'   + '_'     (seen 9x)
merge 4: 'l'     + 'o'     (seen 7x)
merge 5: 'lo'    + 'w'     (seen 7x)
merge 6: 'n'     + 'e'     (seen 6x)
merge 7: 'ne'    + 'w'     (seen 6x)
merge 8: 'new'   + 'est_'  (seen 6x)

Final tokens: ['low _', 'low e r _', 'newest_', 'w i d est_']
```

The tokenizer learned `est_` (a common suffix) and `low` (a common stem) without being told. A new word is tokenized by applying the same merges in order:

Python — Tested: output below is real

```python
def tokenize(word):
    symbols = tuple(word + "_")
    for pair in merges:
        symbols = tuple(merge({symbols: 1}, pair))[0]
    return list(symbols)

for w in ["lowest", "newer", "slow"]:
    print(f"{w:7s} -> {tokenize(w)}")
```

Output

```
lowest  -> ['low', 'est_']
newer   -> ['new', 'e', 'r', '_']
slow    -> ['s', 'low', '_']
```

"lowest" was never in the training data, yet it's built from the known pieces `low` and `est_`. Real tokenizers work the same way at a much larger scale:

Python — Needs: pip install tiktoken transformers

```python
import tiktoken
enc = tiktoken.get_encoding("o200k_base")          # used by recent OpenAI models
ids = enc.encode("Tokenization isn't as simple as splitting on spaces!")
print(ids)
print([enc.decode([i]) for i in ids])

from transformers import AutoTokenizer
bert_tok = AutoTokenizer.from_pretrained("bert-base-uncased")    # WordPiece, a BPE cousin
print(bert_tok.tokenize("Tokenization isn't simple"))
# ['token', '##ization', 'isn', "'", 't', 'simple']   "##" marks a word continuation
```

> **Interview tip:** "Why do LLMs struggle to count the letters in *strawberry*?" Because they never see letters. They see tokens such as `str`, `aw`, `berry`. Tokenization also explains why API pricing is per token and why some languages cost more tokens per sentence than English.

4. Embeddings and positional encoding

Each token ID is turned into a vector by looking up a row in an **embedding matrix** of size (vocabulary size × d_model). These vectors are learned during training, exactly like the embeddings in the Embeddings guide.

But attention (Part 2) looks at all tokens at once, with no built-in sense of order. Without extra information, "dog bites man" and "man bites dog" would look identical. The original Transformer fixed this by **adding** a *positional encoding* to each token's embedding: a unique pattern of sine and cosine waves for every position.

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d}) \quad PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

Python

Tested: output below is real

```python
import numpy as np

def positional_encoding(max_len, d_model):
    pos = np.arange(max_len)[:, None]                # (max_len, 1)
    i = np.arange(0, d_model, 2)[None, :]            # (1, d_model/2)
    angles = pos / (10000 ** (i / d_model))
    pe = np.zeros((max_len, d_model))
    pe[:, 0::2] = np.sin(angles)                     # even dimensions
    pe[:, 1::2] = np.cos(angles)                     # odd dimensions
    return pe

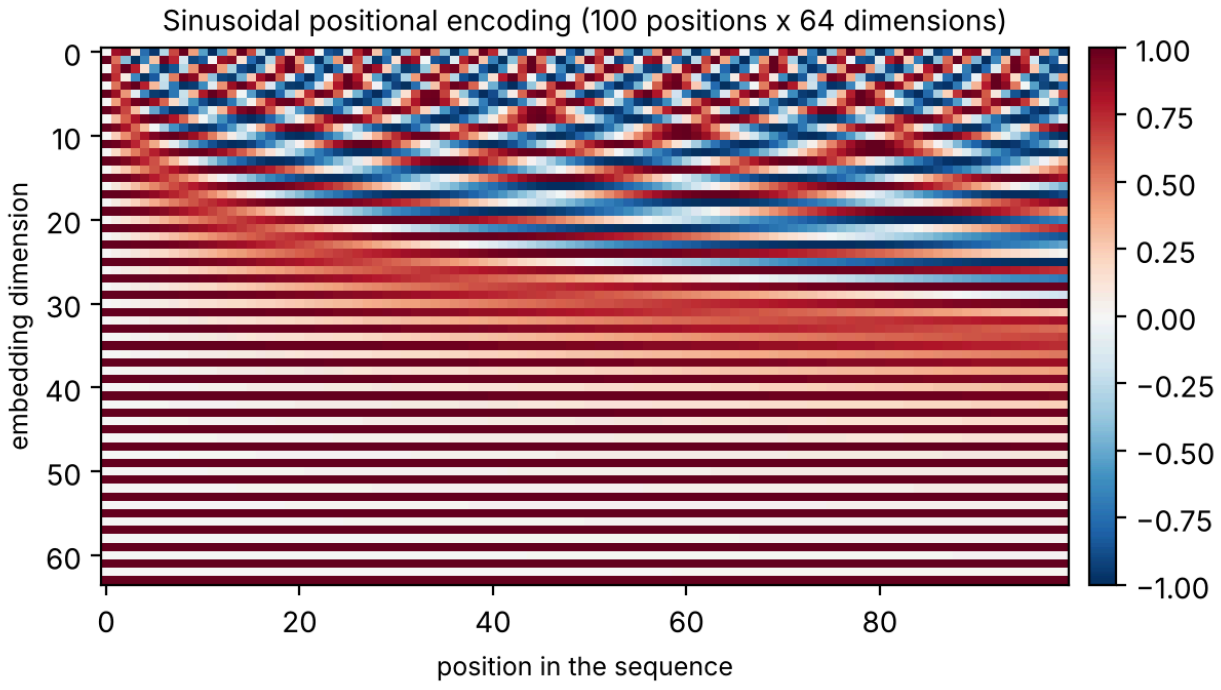
pe = positional_encoding(max_len=50, d_model=16)
print("Shape:", pe.shape)
print("Position 0:", pe[0, :6])
print("Position 1:", pe[1, :6])

# Nearby positions get similar encodings; distant ones less so
cos = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(f"\nsimilarity(pos 10, pos 11) = {cos(pe[10], pe[11]):.3f}")
print(f"similarity(pos 10, pos 40) = {cos(pe[10], pe[40]):.3f}")
```

Output

```
Shape: (50, 16)
Position 0: [0. 1. 0. 1. 0. 1.]
Position 1: [0.841 0.54  0.311 0.95  0.1   0.995]

similarity(pos 10, pos 11) = 0.936
similarity(pos 10, pos 40) = 0.337
```



Low dimensions (top) oscillate quickly and high dimensions slowly, like the hands of a clock, so every position gets a unique fingerprint.

Many models (BERT, GPT-2) instead *learn* a position embedding table. Most modern LLMs use **RoPE** (rotary position embeddings), which rotates the query and key vectors instead. RoPE is covered in Part 5.

Part 2: Attention

5. The intuition

Consider: *"The animal didn't cross the street because it was too tired."* To understand "it", the model must connect it to "animal", not "street". **Self-attention** lets every token gather information from the tokens most relevant to it.

Each token produces three vectors through learned weight matrices:

Vector	Question it answers	Analogy (searching a library)
Query (Q)	What am I looking for?	Your search terms
Key (K)	What do I contain?	The label on each book's spine
Value (V)	What information do I pass on?	The book's actual contents

Each token compares its **query** with every token's **key**. A high match means "pay attention here". The token's new representation is then a **weighted average of the values**, using those match scores as weights.

Here is that idea with hand-made 2D vectors. The first number means "is an animate noun", the second "is a place":

Python — Tested: output below is real

```python
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)       # subtract max for numerical stability
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

tokens = ["The", "animal", "didn't", "cross", "the", "street", "because", "it"]
keys = np.array([[0, 0], [1, 0], [0, 0], [0, 0.2], [0, 0], [0.1, 1], [0, 0], [0.5, 0]])
query_it = np.array([3, 0])           # "it" is looking for an animate noun

weights = softmax(keys @ query_it)
for tok, w in sorted(zip(tokens, weights), key=lambda p: -p[1])[:4]:
    print(f"{tok:8s} {w:.2f} {'#' * int(w * 40)}")
```

Output

```
animal   0.65 ########################
it       0.14 #####
street   0.04 #
The      0.03 #
```

In a real model nobody hand-designs these features. The weight matrices that produce Q, K and V are learned from billions of examples, and the model discovers whatever features are useful.

6. Scaled dot-product attention

The full formula, applied to all tokens at once with matrices:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

1. **QK^T**: every query is compared with every key, giving a (tokens × tokens) score matrix.
2. **÷ $\sqrt{d_k}$**: scale the scores down (explained below).
3. **softmax** on each row turns scores into weights that sum to 1.
4. **× V**: each output is a weighted mix of the value vectors.

Python — Tested: output below is real

```python
rng = np.random.default_rng(0)

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.swapaxes(-1, -2) / np.sqrt(d_k)     # (..., T, T)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)            # blocked positions get ~zero weight
    weights = softmax(scores, axis=-1)
    return weights @ V, weights

T, d_model, d_k = 4, 8, 8           # 4 tokens, 8-dimensional embeddings
X = rng.normal(size=(T, d_model))   # token embeddings (+ positions)
W_q, W_k, W_v = (rng.normal(size=(d_model, d_k)) / np.sqrt(d_model) for _ in range(3))

Q, K, V = X @ W_q, X @ W_k, X @ W_v
out, w = scaled_dot_product_attention(Q, K, V)
print("Q, K, V shapes:", Q.shape, K.shape, V.shape)
print("Output shape:  ", out.shape, "(one new vector per token)")
print("Attention weights (row = token, columns = who it attends to):")
print(w)
print("Each row sums to:", w.sum(axis=1))
```

Output

```
Q, K, V shapes: (4, 8) (4, 8) (4, 8)
Output shape:   (4, 8) (one new vector per token)
Attention weights (row = token, columns = who it attends to):
[[0.344 0.127 0.332 0.198]
 [0.775 0.004 0.053 0.168]
 [0.252 0.278 0.286 0.184]
 [0.216 0.278 0.211 0.295]]
Each row sums to: [1. 1. 1. 1.]
```

Why divide by $\sqrt{d_k}$?

The dot product of two random vectors with *d* dimensions has a variance of about *d*. With d = 512, scores in the dozens are normal, and softmax of large numbers becomes almost one-hot: one token gets all the weight and the gradients for the rest vanish, so training stalls. Dividing by √d brings the variance back to about 1.

Python — Tested: output below is real

```python
for d in [16, 512]:
    q, k = rng.normal(size=(1000, d)), rng.normal(size=(1000, 10, d))
    raw = np.einsum("nd,ntd->nt", q, k)
    for label, s in [("unscaled", raw), ("scaled", raw / np.sqrt(d))]:
        top = softmax(s).max(axis=1).mean()
        print(f"d={d:4d} {label:9s} score std={s.std():5.1f}  average top weight={top:.2f}")
```

Output

```
d=  16 unscaled  score std=  4.0  average top weight=0.72
d=  16 scaled    score std=  1.0  average top weight=0.32
d= 512 unscaled  score std= 22.2  average top weight=0.95
d= 512 scaled    score std=  1.0  average top weight=0.32
```

Without scaling at d = 512, the top token takes almost all of the weight; with scaling, attention stays spread out and learnable.

7. Masking

Two kinds of masks control what each token may attend to.

Causal (look-ahead) mask. GPT-style models generate text left to right, predicting each next token. During training they see the whole sentence at once, so we must stop each position from "cheating" by looking at future tokens. A lower-triangular mask allows each token to see only itself and the tokens before it.

Padding mask. Sequences in a batch are padded to the same length. Padding tokens are meaningless, so real tokens must not attend to them.

Python — Tested: output below is real

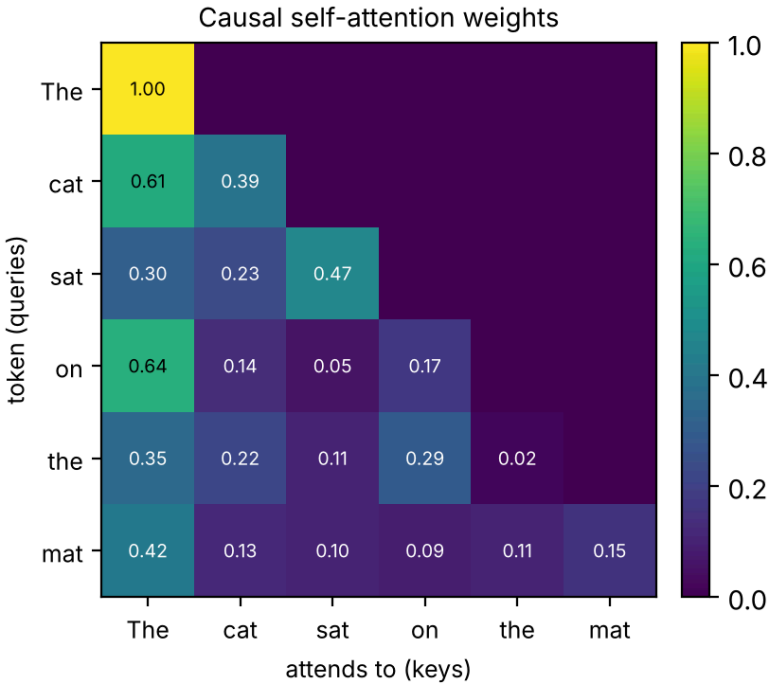
```python
T = 5
causal_mask = np.tril(np.ones((T, T), dtype=bool))
print("Causal mask (1 = allowed):")
print(causal_mask.astype(int))

X5 = rng.normal(size=(T, d_model))
_, w = scaled_dot_product_attention(X5 @ W_q, X5 @ W_k, X5 @ W_v, mask=causal_mask)
print("\nCausal attention weights (no weight above the diagonal):")
print(w)
```

Output

```
Causal mask (1 = allowed):
[[1 0 0 0 0]
 [1 1 0 0 0]
 [1 1 1 0 0]
 [1 1 1 1 0]
 [1 1 1 1 1]]

Causal attention weights (no weight above the diagonal):
[[1.    0.    0.    0.    0.   ]
 [0.376 0.624 0.    0.    0.   ]
 [0.286 0.371 0.343 0.    0.   ]
 [0.399 0.145 0.454 0.001 0.   ]
 [0.263 0.12  0.112 0.401 0.105]]
```



With a causal mask, "The" can only attend to itself, while "mat" can attend to the whole sentence. Weights here come from random matrices; a trained model's patterns are meaningful.

8. Multi-head attention

One attention operation can only capture one kind of relationship at a time. **Multi-head attention** runs several attention operations ("heads") in parallel, each with its own smaller Q, K and V projections. One head might track grammar (which noun does this verb belong to?), another coreference ("it" → "animal"), another nearby words. The head outputs are concatenated and mixed by a final projection $\mathbf{W_o}$.

Python — Tested: output below is real

```python
def multi_head_attention(X, W_q, W_k, W_v, W_o, n_heads, mask=None):
    T, d_model = X.shape
    d_head = d_model // n_heads

    def split_heads(M):                                  # (T, d_model) -> (heads, T, d_head)
        return M.reshape(T, n_heads, d_head).transpose(1, 0, 2)

    Q, K, V = split_heads(X @ W_q), split_heads(X @ W_k), split_heads(X @ W_v)
    out, weights = scaled_dot_product_attention(Q, K, V, mask)   # all heads at once
    out = out.transpose(1, 0, 2).reshape(T, d_model)      # concatenate the heads
    return out @ W_o, weights

d_model, n_heads, T = 16, 4, 6
X = rng.normal(size=(T, d_model))
W_q, W_k, W_v, W_o = (rng.normal(size=(d_model, d_model)) / np.sqrt(d_model) for _ in range(4))

out, weights = multi_head_attention(X, W_q, W_k, W_v, W_o, n_heads)
print("Input: ", X.shape)
print("Attention weights per head:", weights.shape, "(heads, tokens, tokens)")
print("Output:", out.shape, "(same shape as the input)")
```

Output

```
Input:  (6, 16)
Attention weights per head: (4, 6, 6) (heads, tokens, tokens)
Output: (6, 16) (same shape as the input)
```

Key idea: the total cost is about the same as a single large head, because each head works on d_model / n_heads dimensions. You get several specialised "views" of the sentence for free.

Part 3: The full model

9. The Transformer block

Attention lets tokens share information. Each **Transformer block** then adds three more ingredients:

- **Feed-forward network (FFN)**: a small two-layer neural network applied to each token separately. It expands to 4 × d_model, applies a non-linearity (GELU), and projects back. This is where much of the model's "knowledge" is stored; it holds about two-thirds of the parameters.
- **Residual connections**: each sub-layer's output is *added* to its input (x + f(x)). This gives gradients a direct path through deep networks, so 100-layer models can still train.
- **Layer normalisation**: rescales each token's vector to mean 0 and variance 1, keeping values stable. Modern models apply it *before* each sub-layer ("pre-norm").

Python — Tested: output below is real

```python
def layer_norm(x, eps=1e-5):
    return (x - x.mean(-1, keepdims=True)) / np.sqrt(x.var(-1, keepdims=True) + eps)

def gelu(x):
    return 0.5 * x * (1 + np.tanh(np.sqrt(2 / np.pi) * (x + 0.044715 * x ** 3)))

class TransformerBlock:
    def __init__(self, d_model, n_heads, rng):
        s = 1 / np.sqrt(d_model)
        self.n_heads = n_heads
        self.W_q, self.W_k, self.W_v, self.W_o = (rng.normal(size=(d_model, d_model)) * s for _ in
range(4))
        self.W_1 = rng.normal(size=(d_model, 4 * d_model)) * s
        self.W_2 = rng.normal(size=(4 * d_model, d_model)) * s / 2

    def __call__(self, x, mask=None):
        attn, _ = multi_head_attention(layer_norm(x), self.W_q, self.W_k, self.W_v, self.W_o,
self.n_heads, mask)
        x = x + attn                                        # residual connection 1
        x = x + gelu(layer_norm(x) @ self.W_1) @ self.W_2   # residual connection 2 (FFN)
        return x

    def n_params(self):
        return sum(w.size for w in [self.W_q, self.W_k, self.W_v, self.W_o, self.W_1, self.W_2])

blocks = [TransformerBlock(d_model=16, n_heads=4, rng=rng) for _ in range(3)]
x = rng.normal(size=(6, 16))
mask = np.tril(np.ones((6, 6), dtype=bool))
for b in blocks:
    x = b(x, mask)
print("Output after 3 blocks:", x.shape)
print("Parameters per block:", blocks[0].n_params(),
      f"(attention {4 * 16 * 16}, feed-forward {2 * 16 * 64})")
```

Output

```
Output after 3 blocks: (6, 16)
Parameters per block: 3072 (attention 1024, feed-forward 2048)
```

Stack the block N times (12 in GPT-2 small, 96 in GPT-3) and you have the core of a Transformer.

10. Encoders, decoders and encoder-decoders

The original Transformer had two halves: an **encoder** that reads the input, and a **decoder** that generates the output while also attending to the encoder (*cross-attention*). Modern models usually use one half.

Type	Attention	Trained to	Examples	Best for
Encoder-only	Bidirectional (sees left and right)	Fill in masked words	BERT, RoBERTa, DeBERTa	Classification, NER, embeddings, search
Decoder-only	Causal (left only)	Predict the next token	GPT, Llama, Claude, Gemini, Mistral	Text generation, chat, reasoning, code
Encoder-decoder	Encoder bidirectional, decoder causal plus cross-attention	Turn one sequence into another	T5, BART, original Transformer, Whisper	Translation, summarisation, speech-to-text

> **Interview tip:** "Why is BERT better than GPT at classification, and GPT better at generation?" BERT sees context on both sides of every word, so it builds richer representations for understanding. GPT sees only the left context, which is exactly what you need to generate text one token at a time.

11. Build a mini GPT in PyTorch

This is a complete, trainable character-level GPT. It uses every idea from this guide, and it's the same architecture as GPT-2, just smaller. Download a text file to train on, for example the "tiny Shakespeare" dataset:

Terminal

```
curl -O https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
```

Python — Needs: pip install torch

```python
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
device = "cuda" if torch.cuda.is_available() else ("mps" if torch.backends.mps.is_available() else "cpu")

# ---- Data: every character is a token ----
text = open("input.txt", encoding="utf-8").read()
chars = sorted(set(text))
vocab_size = len(chars)
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)

data = torch.tensor(encode(text), dtype=torch.long)
split = int(0.9 * len(data))
train_data, val_data = data[:split], data[split:]

# ---- Hyperparameters (small enough for a laptop) ----
block_size, batch_size = 128, 32        # context length, sequences per batch
d_model, n_heads, n_layers, dropout = 192, 6, 4, 0.1

def get_batch(split):
    d = train_data if split == "train" else val_data
    ix = torch.randint(len(d) - block_size - 1, (batch_size,))
    x = torch.stack([d[i:i + block_size] for i in ix])
    y = torch.stack([d[i + 1:i + block_size + 1] for i in ix])   # targets: shifted by one
    return x.to(device), y.to(device)
```

Python — Needs: pip install torch

```python
class CausalSelfAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.qkv = nn.Linear(d_model, 3 * d_model)     # Q, K and V in one matrix
        self.proj = nn.Linear(d_model, d_model)         # W_o
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(d_model, dim=2)
        # (B, T, C) -> (B, heads, T, head_dim)
        q, k, v = (t.view(B, T, n_heads, C // n_heads).transpose(1, 2) for t in (q, k, v))

        att = (q @ k.transpose(-2, -1)) / math.sqrt(k.size(-1))
        mask = torch.tril(torch.ones(T, T, dtype=torch.bool, device=x.device))
        att = att.masked_fill(~mask, float("-inf"))
        att = self.dropout(F.softmax(att, dim=-1))
        # (Equivalent and faster: F.scaled_dot_product_attention(q, k, v, is_causal=True))

        y = (att @ v).transpose(1, 2).contiguous().view(B, T, C)   # concatenate heads
        return self.dropout(self.proj(y))


class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)
```

```
        self.attn = CausalSelfAttention()
        self.ffn = nn.Sequential(nn.Linear(d_model, 4 * d_model), nn.GELU(),
                                 nn.Linear(4 * d_model, d_model), nn.Dropout(dropout))

    def forward(self, x):
        x = x + self.attn(self.ln1(x))     # pre-norm + residual
        x = x + self.ffn(self.ln2(x))
        return x


class MiniGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(block_size, d_model)    # learned positions, like GPT-2
        self.blocks = nn.Sequential(*[Block() for _ in range(n_layers)])
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        pos = torch.arange(T, device=idx.device)
        x = self.tok_emb(idx) + self.pos_emb(pos)
        logits = self.head(self.ln_f(self.blocks(x)))       # (B, T, vocab_size)
        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
        for _ in range(max_new_tokens):
            logits, _ = self(idx[:, -block_size:])          # crop to the context window
            logits = logits[:, -1, :] / temperature           # only the last position matters
            if top_k is not None:
                v, _ = torch.topk(logits, top_k)
                logits[logits < v[:, [-1]]] = float("-inf")
            probs = F.softmax(logits, dim=-1)
            next_id = torch.multinomial(probs, num_samples=1)
            idx = torch.cat([idx, next_id], dim=1)
        return idx
```

Python Needs: pip install torch

```python
model = MiniGPT().to(device)
print(f"{sum(p.numel() for p in model.parameters()) / 1e6:.2f}M parameters")
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

@torch.no_grad()
def estimate_loss(eval_iters=50):
    model.eval()
    losses = {}
    for split in ("train", "val"):
        losses[split] = torch.stack([model(*get_batch(split))[1] for _ in
range(eval_iters)]).mean().item()
    model.train()
    return losses

for step in range(3001):
    xb, yb = get_batch("train")
    _, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
    if step % 500 == 0:
        l = estimate_loss()
        print(f"step {step:4d}  train loss {l['train']:.3f}  val loss {l['val']:.3f}")

prompt = torch.tensor([encode("ROMEO:")], dtype=torch.long, device=device)
print(decode(model.generate(prompt, max_new_tokens=300, temperature=0.8, top_k=40)[0].tolist()))
```

The loss starts around ln(65) ≈ 4.2 (a random guess among 65 characters) and falls steadily. After a few thousand steps, the model writes text that looks like Shakespeare: invented words, but correct structure, character names and line breaks. Scale up the data, the context length and the number of parameters, and the same code becomes GPT-2.

Part 4: Using pre-trained models

Training from scratch takes huge amounts of data and compute. In practice you start from a **pre-trained** model and use it directly, or fine-tune it. Hugging Face `transformers` gives access to hundreds of thousands of them.

12. Pipelines: one line per task

Python — Needs: pip install transformers torch

```python
from transformers import pipeline

sentiment = pipeline("sentiment-analysis")
print(sentiment("The delivery was late but the food was amazing."))

ner = pipeline("ner", aggregation_strategy="simple")
print(ner("Rakshith studied Data Analytics at Dublin Business School in Ireland."))

zero_shot = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
print(zero_shot("My card was charged twice for one order.",
                candidate_labels=["billing", "delivery", "food quality"]))

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
long_article_text = open("article.txt", encoding="utf-8").read()    # any news article
print(summarizer(long_article_text, max_length=80, min_length=30)[0]["summary_text"])
```

13. Contextual embeddings: the same word, different vectors

The Embeddings guide explained that Word2Vec gives "bank" one vector. BERT gives it a different vector in each sentence, because attention mixes in the surrounding words:

Python — Needs: pip install transformers torch

```python
import torch
from transformers import AutoModel, AutoTokenizer

tok = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")

def word_vector(sentence, word):
    inputs = tok(sentence, return_tensors="pt")
    with torch.no_grad():
        hidden = bert(**inputs).last_hidden_state[0]           # (tokens, 768)
    position = tok.convert_ids_to_tokens(inputs["input_ids"][0]).index(word)
    return hidden[position]

money = word_vector("I deposited cash at the bank.", "bank")
river = word_vector("We had a picnic on the river bank.", "bank")
money2 = word_vector("The bank approved my loan.", "bank")

cos = torch.nn.functional.cosine_similarity
print("bank (money) vs bank (river):", cos(money, river, dim=0).item())    # lower
print("bank (money) vs bank (loan): ", cos(money, money2, dim=0).item())   # higher
```

14. Text generation and sampling strategies

A language model outputs a score (**logit**) for every token in its vocabulary. *How you pick* the next token from those scores changes the output dramatically:

- **Greedy**: always pick the highest. Deterministic, but often repetitive.
- **Temperature**: divide the logits by T before softmax. T < 1 sharpens (safer); T > 1 flattens (more creative, more random).
- **Top-k**: sample only from the k most likely tokens.
- **Top-p (nucleus)**: sample from the smallest set of tokens whose probabilities add up to p.

Python — Tested: output below is real

```python
vocab = np.array(["Paris", "London", "Rome", "Berlin", "banana", "the"])
logits = np.array([4.0, 2.5, 2.0, 1.8, -1.0, 0.5])    # "The capital of France is ..."

print("Greedy:", vocab[np.argmax(logits)])
for t in [0.5, 1.0, 2.0]:
    p = softmax(logits / t)
    print(f"temperature {t}: " + "  ".join(f"{w}={pr:.2f}" for w, pr in zip(vocab, p)))

def top_k_filter(logits, k):
    cutoff = np.sort(logits)[-k]
    return np.where(logits >= cutoff, logits, -np.inf)

def top_p_filter(logits, p):
    probs = softmax(logits)
    order = np.argsort(-probs)
    cumulative = np.cumsum(probs[order])
    keep = order[: np.searchsorted(cumulative, p) + 1]     # smallest set reaching p
    out = np.full_like(logits, -np.inf)
    out[keep] = logits[keep]
    return out

print("\nTop-k (k=3) keeps:  ", vocab[np.isfinite(top_k_filter(logits, 3))].tolist())
print("Top-p (p=0.9) keeps:", vocab[np.isfinite(top_p_filter(logits, 0.9))].tolist())

sample_rng = np.random.default_rng(42)
draws = [str(vocab[sample_rng.choice(len(vocab), p=softmax(top_p_filter(logits, 0.9) / 0.8))]) for _
in range(10)]
print("10 samples (top-p 0.9, temperature 0.8):", draws)
```

Output

```
Greedy: Paris
temperature 0.5: Paris=0.92  London=0.05  Rome=0.02  Berlin=0.01  banana=0.00  the=0.00
temperature 1.0: Paris=0.66  London=0.15  Rome=0.09  Berlin=0.07  banana=0.00  the=0.02
temperature 2.0: Paris=0.41  London=0.19  Rome=0.15  Berlin=0.14  banana=0.03  the=0.07

Top-k (k=3) keeps:   ['Paris', 'London', 'Rome']
Top-p (p=0.9) keeps: ['Paris', 'London', 'Rome']
10 samples (top-p 0.9, temperature 0.8): ['Paris', 'Paris', 'London', 'Paris', 'Paris', 'Rome', 'Paris',
'Paris', 'Paris', 'Paris']
```

Python — Needs: pip install transformers torch

```python
from transformers import AutoModelForCausalLM, AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

inputs = tok("The future of artificial intelligence is", return_tensors="pt")
output = model.generate(**inputs, max_new_tokens=40, do_sample=True,
                        temperature=0.8, top_p=0.9, pad_token_id=tok.eos_token_id)
print(tok.decode(output[0], skip_special_tokens=True))

# A small instruction-tuned chat model works with the same pipeline API
from transformers import pipeline
chat = pipeline("text-generation", model="Qwen/Qwen2.5-0.5B-Instruct")
messages = [{"role": "user", "content": "Explain attention in one sentence."}]
print(chat(messages, max_new_tokens=60)[0]["generated_text"][-1]["content"])
```

15. Fine-tuning BERT for classification

Fine-tuning adds a small classification layer on top of a pre-trained encoder and trains the whole network on your labelled data. A few thousand examples are often enough.

Python — Needs: pip install transformers datasets torch accelerate

```python
import numpy as np
from datasets import load_dataset
from transformers import (AutoModelForSequenceClassification, AutoTokenizer,
                          Trainer, TrainingArguments)

dataset = load_dataset("imdb")                                  # movie reviews: 0 = negative, 1 =
positive
tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")
tokenized = dataset.map(lambda batch: tok(batch["text"], truncation=True), batched=True)
train = tokenized["train"].shuffle(seed=42).select(range(4000))   # small subset for speed
test = tokenized["test"].shuffle(seed=42).select(range(1000))

model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=2)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    return {"accuracy": (np.argmax(logits, axis=-1) == labels).mean()}

args = TrainingArguments(output_dir="imdb-distilbert", num_train_epochs=1, learning_rate=2e-5,
                         per_device_train_batch_size=16, eval_strategy="epoch")
trainer = Trainer(model=model, args=args, train_dataset=train, eval_dataset=test,
                  processing_class=tok, compute_metrics=compute_metrics)
trainer.train()
print(trainer.evaluate())        # typically around 0.88 accuracy after one epoch on 4,000 reviews
```

Part 5: Advanced topics

16. Vision Transformers (ViT)

A Transformer only needs a sequence of vectors, so images work too. **ViT** (Dosovitskiy et al., 2020) cuts an image into fixed-size **patches** (typically 16 × 16 pixels), flattens each patch into a vector, and treats the patches as tokens. A 224 × 224 image becomes a "sentence" of 196 patch tokens.

Python — Tested: output below is real

```
image = rng.random((224, 224, 3))            # height, width, colour channels
patch = 16

n = 224 // patch                              # 14 patches per side
patches = (image.reshape(n, patch, n, patch, 3)
                .transpose(0, 2, 1, 3, 4)     # group pixels by patch
                .reshape(n * n, patch * patch * 3))
print("Image:", image.shape, "->", patches.shape[0], "patch tokens of", patches.shape[1], "numbers
each")

W_embed = rng.normal(size=(patch * patch * 3, 768)) * 0.02    # learned linear projection
tokens = patches @ W_embed
print("Patch embeddings:", tokens.shape, "(then add positions and a [CLS] token, as in BERT)")
```

Output

```
Image: (224, 224, 3) -> 196 patch tokens of 768 numbers each
Patch embeddings: (196, 768) (then add positions and a [CLS] token, as in BERT)
```

Python — Needs: pip install transformers torch pillow

```
from transformers import pipeline
classifier = pipeline("image-classification", model="google/vit-base-patch16-224")
print(classifier("cat.jpg")[:3])
```

17. The KV cache: why generation is fast

When generating, the model produces one token at a time. Naively, every new token would recompute keys and values for the *entire* sequence so far. But keys and values for earlier tokens never change under a causal mask, so we can **cache** them and compute K and V only for the newest token. Every production LLM server does this.

Python — Tested: output below is real

```python
d = 16
Wq, Wk, Wv = (rng.normal(size=(d, d)) / np.sqrt(d) for _ in range(3))
seq = rng.normal(size=(10, d))                    # embeddings of 10 tokens

# Without a cache: recompute everything, then take the last position
def full_recompute(x):
    T = len(x)
    out, _ = scaled_dot_product_attention(x @ Wq, x @ Wk, x @ Wv, np.tril(np.ones((T, T), bool)))
    return out[-1]

# With a cache: store K and V, compute only the new token's q, k, v
k_cache, v_cache, cached_outputs = [], [], []
for t in range(len(seq)):
    x_new = seq[t]
    k_cache.append(x_new @ Wk); v_cache.append(x_new @ Wv)
    q = x_new @ Wq
    weights = softmax(np.array(k_cache) @ q / np.sqrt(d))
    cached_outputs.append(weights @ np.array(v_cache))

same = all(np.allclose(cached_outputs[t], full_recompute(seq[: t + 1])) for t in range(len(seq)))
print("Cached generation gives identical outputs:", same)
print("K/V projections computed without cache:", sum(range(1, 11)) * 2, " with cache:", 10 * 2)
```

Output

```
Cached generation gives identical outputs: True
K/V projections computed without cache: 110  with cache: 20
```

The catch is memory: the cache stores K and V for every layer, head and token. For long conversations it can be larger than the model itself, which is why techniques like **grouped-query attention** (GQA) share K and V across groups of heads to shrink it.

18. The cost of long context and efficient attention

Attention compares every token with every other token, so cost grows with the **square** of the sequence length:

Python — Tested: output below is real

```python
print(f"{'tokens':>8s} {'attention scores':>18s} {'memory (one head, fp16)':>24s}")
for T in [1_000, 8_000, 32_000, 128_000]:
    print(f"{T:8,d} {T * T:18,d} {T * T * 2 / 1e9:21.2f} GB")
```

Output

```
  tokens   attention scores  memory (one head, fp16)
   1,000          1,000,000                  0.00 GB
   8,000         64,000,000                  0.13 GB
  32,000      1,024,000,000                  2.05 GB
 128,000     16,384,000,000                 32.77 GB
```

Doubling the context quadruples the work. Several techniques address this:

Technique	Idea
FlashAttention	Computes exact attention in small tiles that fit in fast GPU memory, never storing the full T × T matrix. Same result, much faster and less memory. Used through `F.scaled_dot_product_attention` in PyTorch.
Grouped/multi-query attention (GQA, MQA)	Many query heads share fewer key/value heads, shrinking the KV cache. Used in Llama, Mistral and most modern LLMs.
Sliding-window attention	Each token attends only to the previous *w* tokens; stacking layers still lets information travel far.
Sparse / linear attention, state-space models	Approximate or replace attention to get closer to linear cost (Mamba and hybrid models).

19. Rotary position embeddings (RoPE)

Most modern LLMs (Llama, Mistral, Qwen, Gemma and others) use **RoPE** instead of adding positional encodings. RoPE **rotates** each pair of dimensions in the query and key vectors by an angle proportional to the token's position. The beautiful property: the dot product between a query at position *m* and a key at position *n* depends only on the **distance m − n**, so the model naturally learns *relative* positions.

Python — Tested: output below is real

```python
def rope(x, position, base=10000):
    d = x.shape[-1]
    theta = position / base ** (np.arange(0, d, 2) / d)    # one angle per pair of dimensions
    cos, sin = np.cos(theta), np.sin(theta)
    x1, x2 = x[0::2], x[1::2]
    out = np.empty_like(x)
    out[0::2] = x1 * cos - x2 * sin                         # rotate each (x1, x2) pair
    out[1::2] = x1 * sin + x2 * cos
    return out

q, k = rng.normal(size=8), rng.normal(size=8)
for m, n in [(5, 3), (12, 10), (105, 103)]:
    print(f"query at {m:3d}, key at {n:3d} (distance {m - n}): score = {rope(q, m) @ rope(k, n):.4f}")
print(f"query at  12, key at   3 (distance 9): score = {rope(q, 12) @ rope(k, 3):.4f}")
```

Output

```
query at   5, key at   3 (distance 2): score = 1.5978
query at  12, key at  10 (distance 2): score = 1.5978
query at 105, key at 103 (distance 2): score = 1.5978
query at  12, key at   3 (distance 9): score = 1.6639
```

The same distance gives the same score wherever the pair sits in the text, and rotation never changes a vector's length, so values stay stable.

20. Parameter-efficient fine-tuning: LoRA

Fully fine-tuning a 7-billion-parameter model means updating, and storing, 7 billion numbers. **LoRA** (Low-Rank Adaptation) freezes the original weights and learns a small update instead: a weight matrix W (d × d) gets an added term **B × A**, where A is (r × d) and B is (d × r), with a small rank r such as 8. Only A and B are trained.

Python — Tested: output below is real

```python
d, r = 4096, 8                        # a typical LLM hidden size and LoRA rank
full, lora = d * d, 2 * d * r
print(f"Full fine-tuning of one {d}x{d} matrix: {full:,} parameters")
print(f"LoRA (rank {r}):                        {lora:,} parameters  ({lora / full:.2%})")

# B starts at zero, so the model is unchanged before training begins
d_small = 64
W = rng.normal(size=(d_small, d_small))
A = rng.normal(size=(r, d_small)) * 0.01
B = np.zeros((d_small, r))
x = rng.normal(size=d_small)
print("Output unchanged at the start:", np.allclose(x @ W.T, x @ (W + B @ A).T))
```

Output

```
Full fine-tuning of one 4096x4096 matrix: 16,777,216 parameters
LoRA (rank 8):                         65,536 parameters  (0.39%)
Output unchanged at the start: True
```

Python — Needs: pip install transformers peft torch

```python
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=2)
config = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.1, task_type="SEQ_CLS",
                    target_modules=["q_lin", "v_lin"])     # DistilBERT's query and value layers
model = get_peft_model(model, config)
model.print_trainable_parameters()     # about 1% of the parameters are trainable
# Train with the same Trainer code as Section 15.
```

QLoRA goes further: it loads the frozen base model in 4-bit precision and trains LoRA adapters on top. That's how people fine-tune 7–70B models on a single GPU.

21. What makes modern LLMs different

The core architecture has barely changed since GPT-2. The progress came from:

- **Scale**: more parameters, much more data and compute, following *scaling laws* that predict how loss falls as each grows.
- **Better training data**: careful filtering, deduplication, code and synthetic data.
- **Post-training**: instruction tuning, then preference optimisation (RLHF, DPO) to make models helpful and safe, and reinforcement learning for reasoning.
- **Mixture of Experts (MoE)**: many feed-forward "experts" per layer, with a router sending each token to only a few. The model has huge capacity, but each token only uses a fraction of it.
- **Efficiency**: FlashAttention, GQA, RoPE, KV caching and quantisation (int8 and 4-bit weights) make serving affordable.

Summary and interview questions

Cheat sheet

Concept	One-line summary
Token	A subword unit; models see token IDs, not characters
BPE	Builds a vocabulary by merging the most frequent adjacent pairs
Positional encoding	Adds order information that attention otherwise lacks
Q, K, V	What I'm looking for, what I contain, what I pass on
Scaled dot-product attention	$\text{softmax}(QK^T / \sqrt{d})\ V$
√d scaling	Keeps softmax from saturating, so gradients survive
Causal mask	Stops a token from seeing the future during training
Multi-head attention	Several attention patterns in parallel, at the cost of one
Transformer block	Attention + FFN, each wrapped in layer norm and a residual
Encoder / decoder	Bidirectional understanding / left-to-right generation
Temperature, top-k, top-p	Control the randomness of generation
KV cache	Reuse past keys and values so each new token is cheap
FlashAttention, GQA	Faster exact attention; smaller KV cache
RoPE	Rotations that encode relative position
LoRA	Train a tiny low-rank update instead of all the weights

Interview questions to practise

1. **Explain self-attention to a non-technical person.** Every word looks at every other word and decides how much each one matters for understanding it, then blends in information from the important ones.
2. **What is the complexity of self-attention and why does it matter?** $O(T^2)$ in sequence length, for both compute and memory. It limits context length and drives techniques like FlashAttention and sliding windows.
3. **Why do Transformers need positional information?** Attention treats its input as an unordered set; without positions, word order would be invisible.
4. **What's the difference between BERT and GPT?** Encoder with bidirectional attention trained on masked words, versus decoder with causal attention trained on next-token prediction.
5. **Why use multiple heads?** Each head can specialise in a different relationship (syntax, coreference, nearby words), with no extra cost overall.
6. **What do residual connections and layer norm do?** Residuals give gradients a direct path through deep stacks; layer norm keeps activations at a stable scale.
7. **How does a KV cache speed up inference, and what does it cost?** It avoids recomputing past keys and values, making each new token $O(T)$ instead of $O(T^2)$, at the price of memory that grows with

context length.

8. **How would you adapt an LLM to your company's data cheaply?** Start with RAG for knowledge; if the model's style or task behaviour must change, fine-tune with LoRA or QLoRA on a few thousand examples.